

Data Parallelism

Many workers, one model, the same step · video 2:27

This lesson answers one question: how do many workers train one model together? In **data parallelism**, every worker holds a full copy of the model and processes its own slice of the batch; an **all-reduce** averages the gradients so every copy takes the same step. A real run with 4 processes matches a single process to within float rounding for 200 steps. The costs: exchanging every gradient every step, and holding the whole model, gradients and optimizer state on every worker.

How to use this guide: master one concept before moving to the next.

1. Watch the whole video once, without stopping.
2. Then take the concepts **in order**. For each one: read the short explanation, look at the picture from the video (▶ is the time in the video), and answer every question in its *Check* without looking anything up.
3. Check your answers in the **Answers** section at the end (page 5).
4. **Pass mark: every question in the check correct.** If you missed one, rewatch that concept's video segment, reread the explanation, and redo only the questions you missed. Tick the concept off below, then move on.

It is strongly suggested that you **master each concept before starting the next one**. Each concept builds on the previous ones.

Concepts in this lesson

CONCEPT	VIDEO SEGMENT	QUESTIONS	MASTERED
1. Copies, slices, and all-reduce	▶ 0:08-0:49	2	☐
2. Exactly the same training	▶ 0:49-1:17	2	☐
3. The costs	▶ 1:17-2:25	2	☐

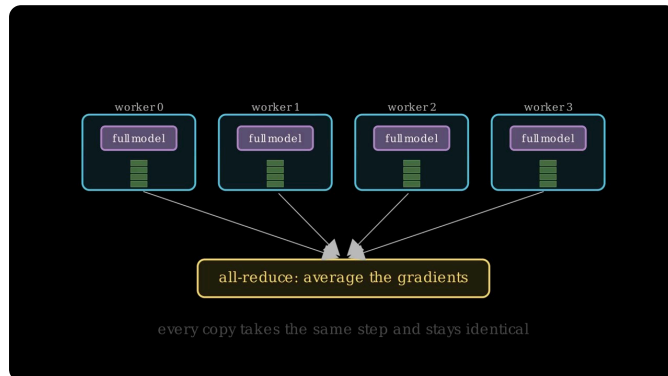
Before you start: Deep Dive episodes 19 (AdamW) and 21 (batch size and gradient accumulation).
Code: `code/d25_data_parallel` (PyTorch only; starts 4 local processes; a few minutes on a CPU).

CONCEPT 1 OF 3

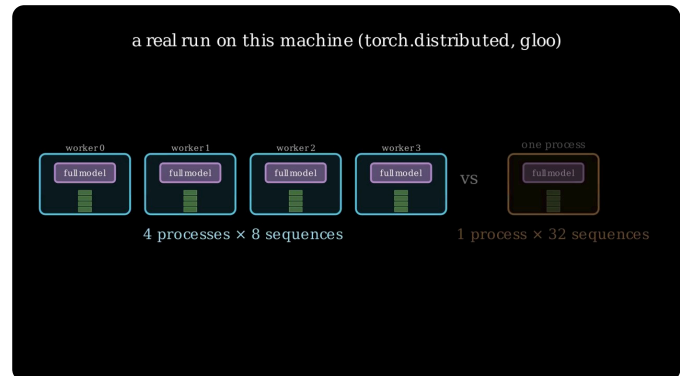
Copies, slices, and all-reduce

▶ Video 0:08–0:49

Large models are trained on thousands of GPUs. The simplest way to use many workers is **data parallelism**: every worker holds a full copy of the model and takes its own slice of the batch. Each computes gradients on its slice; then the workers **average their gradients** (an **all-reduce**), so every copy takes exactly the same step and they stay identical.



▶ 0:35 Each worker: a full model copy and a slice of the batch; all-reduce averages the gradients.



▶ 0:48 A real run: 4 processes × 8 sequences vs 1 process × 32 sequences.

The episode runs it for real on one machine: 4 worker processes talking through PyTorch's distributed library (the “gloo” backend; GPUs use “nccl”), each with 8 sequences, compared with one process that takes all 32.

KEY IDEA

Data parallelism splits the batch, not the model; an all-reduce keeps the copies in sync.

Check 1: Copies, slices, and all-reduce

Answer all 2 questions before looking at the answers. Pass mark: all correct.

1.1 SHORT ANSWER

Why must every worker start from the same weights?

1.2 TRUE OR FALSE

“In data parallelism, each worker holds a quarter of the model.”

True ☐ False ☐

CONCEPT 2 OF 3

Exactly the same training

▶ Video 0:49-1:17

The first gradient matches the single process to within 1.5×10^{-8} , and over 200 steps the losses never differ by more than 4.8×10^{-7} : 4.3335 at the start, 2.3006 at the end, in both runs. Why exact? The batch loss is an average, and the average of four averages over equal slices is the average over all 32. Splitting the work changes nothing in the math (only the order of float additions).

```
first gradient: largest difference  $1.5 \times 10^{-8}$ 
losses over 200 steps: largest difference  $4.8 \times 10^{-7}$ 
step 1: 4.3335 vs 4.3335
step 200: 2.3006 vs 2.3006
```

```
mean(mean of 8, mean of 8, mean of 8, mean of 8) = mean of 32
```

splitting the work changes nothing in the math

▶ 1:04 First gradient within 1.5×10^{-8} ; losses within 4.8×10^{-7} for 200 steps: 4.3335 → 2.3006 in both runs.

▶ 1:16 The average of four averages over equal slices is the average over all 32.

KEY IDEA

With equal slices, data-parallel training is mathematically the same as one big batch.

Check 2: Exactly the same training

Answer all 2 questions before looking at the answers. Pass mark: all correct.

2.1 HANDS-ON CODE

Try it yourself. Change the script to 2 workers × 16 sequences (and fewer steps). Does it still match?

```
WORLD, PER_WORKER, STEPS = 2, 16, 50
```

2.2 SHORT ANSWER

What would break if one worker got 4 sequences and another 12, and DDP still took a plain average of their gradients?

CONCEPT 3 OF 3

The costs

▶ Video 1:17-2:25

Communication. Every step, every worker must exchange all its gradients: 3.1 MiB for the tiny model, 0.5 GiB for GPT-2, 26.1 GiB for a 7-billion-parameter model (float32). The usual method is a **ring all-reduce**: workers pass chunks to their neighbor around a ring, and each sends about $2 \times (N - 1)/N \times$ the gradient size, however many workers there are (4.7 MiB for the tiny model with 4 workers). Libraries start sending during the backward pass, to hide the wait.

Memory. Every worker needs the whole model: with Adam in float32 that is 16 bytes per parameter (weights, gradients, and Adam's m and v, 4 bytes each). For 7 billion parameters: 112 GB, more than one GPU holds. The next episodes split the model itself. In code, data parallelism is two lines: start the process group, wrap the model in `DistributedDataParallel`.

gradients every worker exchanges, every step (float32)

our tiny model (818,241 parameters)	3.1 MiB
GPT-2 small	0.5 GiB
7 billion parameters	26.1 GiB

▶ 1:32 Gradients exchanged every step: 3.1 MiB (tiny model), 0.5 GiB (GPT-2), 26.1 GiB (7 billion parameters).

the catch: every worker holds the whole model

weights 4 bytes	gradients 4 bytes	Adam m 4 bytes	Adam v 4 bytes
--------------------	----------------------	-------------------	-------------------

16 bytes per parameter

7 billion parameters → 112 GB: more than one GPU holds

next: split the model itself

▶ 2:07 Every worker holds 16 bytes per parameter with Adam: 7 billion parameters → 112 GB.

KEY IDEA

Data parallelism scales compute, not memory: each worker still needs room for the whole training state.

Check 3: The costs

Answer all 2 questions before looking at the answers. Pass mark: all correct.

3.1 CALCULATION

With 8 workers, how many GiB does each send per step in a ring all-reduce of GPT-2's 0.46 GiB of gradients?

3.2 CALCULATION

How much memory does GPT-2 small's training state take per worker (16 bytes per parameter)?

ANSWERS

Answers and explanations

Mark each answer right or wrong. If a concept has any wrong answer, rewatch its segment (▶) and redo the questions you missed before moving on.

1. Copies, slices, and all-reduce · rewatch ▶ 0:08-0:49

1.1 They all apply the same averaged update; if they started differently, they would stay different, and averaging gradients from different models would not make sense. The code seeds every worker identically (`torch.manual_seed(0)`); DDP also broadcasts rank 0's weights at the start.

1.2 False. Each holds the whole model and a quarter of the batch.

2. Exactly the same training · rewatch ▶ 0:49-1:17

2.1 Yes: first gradient within 1.5×10^{-8} , losses within 4.8×10^{-7} (step 50: 2.5632 in both). Any equal split of the 32 sequences gives the same average gradient.

2.2 The average would weight the 4 sequences as much as the 12, so it would no longer equal the batch average. Equal slices make the mean of means equal the overall mean.

3. The costs · rewatch ▶ 1:17-2:25

3.1 About 0.81 GiB. $2 \times 7/8 \times 0.4636 \text{ GiB} \approx 0.81$.

3.2 About 2.0 GB (1.85 GiB). $124,439,808 \times 16 \text{ bytes} \approx 1.99 \times 10^9 \text{ bytes}$.